Lab 1: Logical Agents with Propositional Logic

This lab consists of two parts:

- **Part 1:** Implement propositional logic, evaluate sentences in a given model, and query a knowledge base.
- **Part 2:** Use the propositional logic and query function from Part 1 to create a knowledge-based agent that traverses the Wumpus World.

Part 2 builds directly on Part 1, so make sure Part 1 is implemented and tested before you start Part 2.

Part 1: Making Propositional Logic (PL) and Querying PL Knowledge base

In this part you will implement propositional logic which will be the foundation of our AI agent. Initially it might seem strange to implement propositional logic, since Python already has it implemented in the simple `not`, `and`, `or`, and `==` operators (implication can be derived from these as well). While it's true that any propositional logic statement can be evaluated using Python, these statements cannot be stored in their unevaluated form (which is needed to form a Knowledge Base (KB)). Additionally, we cannot create variables without assigning them a value in standard Python, but when we later want to iterate over all possible models the variables will need to take on many possible values.

So we will need to implement our own version of propositional logic in Python. For variables we will use strings to represent them and later we can use dictionaries to represent models and then look up the value of each variable in that model. For True and False we will simply use the existing `True` and `False` values in Python. For all the operators we will implement classes that contain the logic sentences they operate on.

> **Task 1:** Given a skeleton for the "and" operator, you need to expand it to each of the other operators.

> **Task 2:** Extend the operators so that they can take more than two sentences, e.g. `And(s1,s2,s3, ...)`. Also implement a function that, given a binary operator and a list of sentences, chains them together (e.g. `f(And, [s1, s2, s3]) -> And(s1, And(s2, s3))`).

For some operators this isn't necessary or doesn't make sense. For example, `Not` applies to only one sentence. Implication is not associative, so `(p→q)→r` is not equivalent to `p→(q→r)`.

In [1]:
```
class And:
    def __init__(self, *sentences):
        self.sentences = sentences

    def __repr__(self):
        return "(" + " ∧ ".join(map(str, self.sentences)) + ")"

    def __eq__(self, other):
        return type(self) == type(other) and self.__dict__ == other.__dict__

    def __hash__(self):
        return hash((type(self), tuple(sorted(self.__dict__.items()))))


class Or:
    def __init__(self, *sentences):
        self.sentences = sentences

    def __repr__(self):
        return "(" + " ∨ ".join(map(str, self.sentences)) + ")"
    def __eq__(self, other):
        return type(self) == type(other) and self.__dict__ == other.__dict__

    def __hash__(self):
        return hash((type(self), tuple(sorted(self.__dict__.items()))))


# Equivalent (<=>)
class Equals:
    def __init__(self, A, B):
        self.A = A
        self.B = B

    def __repr__(self):
        return f"({self.A} ⇔ {self.B})"

    def __eq__(self, other):
        return type(self) == type(other) and self.__dict__ == other.__dict__

    def __hash__(self):
        return hash((type(self), tuple(sorted(self.__dict__.items()))))


class Implies:
    def __init__(self, A, B):
        self.A = A
        self.B = B

    def __repr__(self):
        return f"({self.A} ⇒ {self.B})"

    def __eq__(self, other):
        return type(self) == type(other) and self.__dict__ == other.__dict__

    def __hash__(self):
        return hash((type(self), tuple(sorted(self.__dict__.items()))))


class Not:
```

```
    def __init__(self, A):
        self.A = A

    def __repr__(self):
        return f"¬({self.A})"

    def __eq__(self, other):
        return type(self) == type(other) and self.__dict__ == other.__dict__

    def __hash__(self):
        return hash((type(self), tuple(sorted(self.__dict__.items()))))


def chain(operator, sentences):
    """
    Chains a list of sentences using a binary operator.
    Example:
    chain(And, ["P", "Q", "R"]) ->
    (P ∧ (Q ∧ R))
    """

    if len(sentences) < 2:
        raise ValueError("At least two sentences are required.")

    result = sentences[-1]

    for sentence in reversed(sentences[:-1]):
        result = operator(sentence, result)

    return result
```

In [2]:

```
P = "P"
Q = "Q"
R = "R"
S = "S"

# Multiple arguments
expr1 = And(P, Q, R, S)
print(expr1)

expr2 = Or(P, Q, R)
print(expr2)

# Binary operators
expr3 = Implies(P, Q)
print(expr3)

expr4 = Equals(P, Q)
print(expr4)

expr5 = Not(P)
print(expr5)

# Chained binary operators
expr6 = chain(And, [P, Q, R])
print(expr6)

expr7 = chain(Or, [P, Q, R, S])
print(expr7)
```

```
(P ∧ Q ∧ R ∧ S)
(P ∨ Q ∨ R)
(P ⇒ Q)
(P ⇔ Q)
¬(P)
(P ∧ (Q ∧ R))
(P ∨ (Q ∨ (R ∨ S)))
```

Evaluating propositional sentences in a given model

Being able to form sentences in propositional logic is not very useful if we cannot actually evaluate them. A start of the function can be found below, but it is missing most things.

> **Task 3:** Implement a function `evaluate` that takes a sentence in propositional logic and a dictionary with the truth values of the different variables in a given model and returns the truth value of the sentence in that model.

> **Task 4:** Extend `evaluate` to handle unknown values (`None`): it should return `True` if the sentence is true, `False` if it is false, and `None` if the answer cannot be known because some variables are unknown.

Note that the function for replacing the variables with their correct truth-value will currently give an error if a variable is missing, but we want it to return `None` to represent an unknown value instead.

This is done to make evaluation of large models more efficient down the line. We add the concept of an unknown or uninteresting truth value using the Python `None` -type. The idea with unknown values is that instead of iterating over every value for every variable to evaluate a model, we can decide to only iterate over some variables and set the others as unknown. However if an important variable is set to unknown we might not be able to get the correct answer. In order to protect against this the `evaluate` function will return `True` if the sentence is true, `False` if its false, and `None` if we cannot know since some variables are unknown to us. In order to deal with the `None` values you will have to think what the answer to many different operators should be. For example, we know that `And(False, None)` should be `False` since `And` always returns `False` when at least one sub-sentence is `False` . You will also need to experiment with Python to see if the Python operators give the correct behavior. Does `False == None` return `None` ? If not, how should you deal with it?

In [3]:
```python
def evaluate(sentence, variables):
    """
    Evaluates the truth value of a propositional logic sentence.

    Args:
        sentence: Boolean, None, variable string, or logical operator
        variables: Dictionary mapping variable names to True/False

    Returns:
```

```
    True, False, or None
"""

# Boolean constants and None
if sentence is True or sentence is False or sentence is None:
    return sentence

# Variable lookup
if isinstance(sentence, str):
    return variables.get(sentence, None)

# NOT
elif isinstance(sentence, Not):
    a = evaluate(sentence.A, variables)

    if a is None:
        return None

    return not a

# AND
elif isinstance(sentence, And):
    values = [evaluate(s, variables) for s in sentence.sentences]

    if False in values:
        return False

    if None in values:
        return None

    return True

# OR
elif isinstance(sentence, Or):
    values = [evaluate(s, variables) for s in sentence.sentences]

    if True in values:
        return True

    if None in values:
        return None

    return False

# IMPLIES (A ⇒ B)
elif isinstance(sentence, Implies):
    a = evaluate(sentence.A, variables)
    b = evaluate(sentence.B, variables)

    if a is False:
        return True

    if a is True:
        return b

    # a is None
    if b is True:
        return True

    return None
```

```
    # EQUIVALENCE (A ⇔ B)
    elif isinstance(sentence, Equals):
        a = evaluate(sentence.A, variables)
        b = evaluate(sentence.B, variables)

        if a is None or b is None:
            if a == b and a is not None:
                return True
            return None

        return a == b

    raise ValueError(f"Unknown sentence type: {type(sentence)}")
```

In [4]:
```
model = {
    "P": True,
    "Q": False
}

print(evaluate(And("P", "Q"), model))
# False

print(evaluate(Or("P", "Q"), model))
# True

print(evaluate(Not("Q"), model))
# True

print(evaluate(Implies("P", "Q"), model))
# False

print(evaluate(Implies("Q", "P"), model))
# True

print(evaluate(Equals("P", "Q"), model))
# False

print(evaluate(And("P", "R"), model))
# None (R is unknown)

print(evaluate(Or("P", "R"), model))
# True

print(evaluate(Or("Q", "R"), model))
# None
```

```
False
True
True
False
True
False
None
True
None
```

Testing the evaluate function

> **Task 5:** Implement and evaluate the sentences of an example below for its test model, and check that you get the expected results.

We have now created a propositional logic grammar and a function to evaluate the truth value of sentences for a given model. In order to make sure everything is correct we introduce a testing space below and ask you to implement and evaluate a number of sentences for a given model in order to see that you get the correct result.

Below you are given a model and are then supposed to implement and run evaluation on the sentences below, with the given expected results:

Example 1

- $rain \leftrightarrow \neg sunny$ is `True`
- $rain \rightarrow umbrella$ is `True`
- $(\neg sunny \rightarrow \neg umbrella) \leftrightarrow False$ is `True`
- $(sunny \vee rain) \rightarrow play$ is `None`
- $sunny \rightarrow play$ is `True`
- $\neg rain \leftrightarrow play$ is `None`
- $\neg play$ is `None`
- $(sunny \wedge rain \wedge snow) \leftrightarrow False$ is `True`

Example 2

- $rain \leftrightarrow \neg sunny$ is `True`
- $umbrella \leftrightarrow rain$ is `True`
- $\neg cloudy \leftrightarrow False$ is `True`
- $(sunny \vee rain) \rightarrow storm$ is `None`
- $\neg umbrella \leftrightarrow False$ is `True`
- $sunny \rightarrow cloudy$ is `True`
- $(rain \wedge cloudy) \rightarrow storm$ is `None`
- $(rain \vee cloudy \vee storm)$ is `True`

Example 3

- $windy \leftrightarrow \neg humid$ is `True`
- $power \rightarrow alert$ is `False`
- $(\neg humid \rightarrow \neg power) \leftrightarrow False$ is `True`
- $(windy \vee humid) \rightarrow generator$ is `None`
- $alert \rightarrow windy$ is `True`
- $(\neg alert) \leftrightarrow False$ is `False`
- $\neg generator$ is `None`
- $(windy \wedge humid \wedge alert) \leftrightarrow False$ is `True`

The test model for each example is given below. You are free to test with any (or all) of the examples.

In [5]:
```
# ------------------------------------------------
# Example 1
```

```
# ------------------------------------------------
model1 = {
    "rain": True,
    "sunny": False,
    "umbrella": True,
    "play": None,
    "snow": False
}

print("EXAMPLE 1")
print(evaluate(Equals("rain", Not("sunny")), model1))
# Expected: True

print(evaluate(Implies("rain", "umbrella"), model1))
# Expected: True

print(evaluate(
    Equals(
        Implies(Not("sunny"), Not("umbrella")),
        False
    ),
    model1
))
# Expected: True

print(evaluate(
    Implies(
        Or("sunny", "rain"),
        "play"
    ),
    model1
))
# Expected: None

print(evaluate(
    Implies("sunny", "play"),
    model1
))
# Expected: True

print(evaluate(
    Equals(
        Not("rain"),
        "play"
    ),
    model1
))
# Expected: None

print(evaluate(
    Not("play"),
    model1
))
# Expected: None

print(evaluate(
    Equals(
        And("sunny", "rain", "snow"),
        False
    ),
```

```
    model1
))
# Expected: True


# ------------------------------------------------
# Example 2
# ------------------------------------------------
model2 = {
    "rain": True,
    "sunny": False,
    "umbrella": True,
    "cloudy": True,
    "storm": None
}

print("\nEXAMPLE 2")
print(evaluate(Equals("rain", Not("sunny")), model2))
# Expected: True

print(evaluate(Equals("umbrella", "rain"), model2))
# Expected: True

print(evaluate(
    Equals(
        Not("cloudy"),
        False
    ),
    model2
))
# Expected: True

print(evaluate(
    Implies(
        Or("sunny", "rain"),
        "storm"
    ),
    model2
))
# Expected: None

print(evaluate(
    Equals(
        Not("umbrella"),
        False
    ),
    model2
))
# Expected: True

print(evaluate(
    Implies("sunny", "cloudy"),
    model2
))
# Expected: True

print(evaluate(
    Implies(
        And("rain", "cloudy"),
        "storm"
```

```
    ),
    model2
))
# Expected: None

print(evaluate(
    Or("rain", "cloudy", "storm"),
    model2
))
# Expected: True


# ------------------------------------------------
# Example 3
# ------------------------------------------------
model3 = {
    "windy": True,
    "humid": False,
    "power": True,
    "alert": False,
    "generator": None
}

print("\nEXAMPLE 3")
print(evaluate(
    Equals(
        "windy",
        Not("humid")
    ),
    model3
))
# Expected: True

print(evaluate(
    Implies(
        "power",
        "alert"
    ),
    model3
))
# Expected: False

print(evaluate(
    Equals(
        Implies(
            Not("humid"),
            Not("power")
        ),
        False
    ),
    model3
))
# Expected: True

print(evaluate(
    Implies(
        Or("windy", "humid"),
        "generator"
    ),
    model3
```

```
))
# Expected: None

print(evaluate(
    Implies(
        "alert",
        "windy"
    ),
    model3
))
# Expected: True

print(evaluate(
    Equals(
        Not("alert"),
        False
    ),
    model3
))
# Expected: False

print(evaluate(
    Not("generator"),
    model3
))
# Expected: None

print(evaluate(
    Equals(
        And("windy", "humid", "alert"),
        False
    ),
    model3
))
# Expected: True
```

```
EXAMPLE 1
True
True
True
None
True
None
None
True

EXAMPLE 2
True
True
True
None
True
True
None
True

EXAMPLE 3
True
False
True
None
True
False
None
True
```

Querying for possible models from a Knowledge Base

> **Task 6:** Implement the `query` function (skeleton below). Given a query sentence, a knowledge base, and a list (or set) of relevant variables, it returns the "valid" models where the query is `True`, `False`, or `None`.

Now we are ready to create a knowledge base. Making a knowledge base is not that difficult, it's simply a collection of logical sentences that are known (or presumed) to be true in a given environment.

To create this collection we will use the Python `set` data structure. The reason is that sets can only contain one of each element so we don't risk adding the same sentence multiple times. Sets can be created by listing a number of elements in curly brackets (e.g. `{s1, s2}`), and elements can be added and removed with the `.add()` and `.remove()` functions. Note that an empty set must be created with `set()`, since `{}` creates an empty dictionary.

Hint: For a set to recognize that two sentences are the same (e.g. two separately created `Not("rain")`), your operator classes need `__eq__` and `__hash__` methods. Without them every new object counts as a different sentence, and `.remove()` will not find it.

The more difficult part is then creating a function for querying the knowledge base for whether a new sentence is true or not. One simple way to do this query is to test every

possible model (every possible combination of truth values for the variables). For each model we figure out whether the query sentence is True or False, we then also check that all sentences in the knowledge base are True for that model. Since we "know" that each sentence in the knowledge base should be true, any model that doesn't align with this can be discarded as an "invalid" model of our environment. For all "valid" models we then return whether the query is True or False in that model.

Of course many environments have a huge amount of variables which would then require us to check a huge number of models. If we have n variables in the environment we would need to check each sentence 2^n times. However, not every variable is necessarily relevant for our query sentence.

Using our knowledge of the environment we might be able to deduce the minimum number of variables that are relevant for our query and only test all combinations of them. This is why we implemented the concept of unknown (`None`) values earlier. Due to this we also consider a model "valid" if a sentence in the knowledge base evaluates as unknown. This comes with a risk of course: if we mistakenly exclude a relevant variable in our query a model that should be "invalid" might be considered "valid" since a False evaluation might now be unknown. Additionally, we need to return all "valid" models where the query is unknown.

We can then use this query function to figure out things about the environment. If a query sentence only gives "valid" models in which it is True, then we know the sentence is True in the environment. Conversely, if all returned models are False, the sentence is False in the environment. If it returns some True models and some False models we know that, given what we know of the environment, we cannot yet tell whether the sentence is True or not (in some advanced scenarios we might be able to use the number of True/False models to estimate the likelihood of the query being True/False in the actual environment). If some of the returned models are unknown models, we know that there are relevant variables that were not considered in the query. This likely means we have an error in the code that determines which variables should be relevant for a query. Note that many variables that are not in the query sentence could still be relevant.

Finally, if the function returns no valid models at all, this means that there are contradictions in the knowledge base and something has gone wrong in its construction.

Below you will find the skeleton of the query function for you to complete.

In [6]:
```
import itertools
# Hint: You can use the ```itertools.product()``` function to generate a list of
# [1,2,3]*3 creates a list where the elements of the list are repeated 3 times [
# function(*[1,2,3]) is the same as function(1, 2, 3)
list(itertools.product(*[(1,2,3)]*3))
```

Out[6]:
```
[(1, 1, 1),
 (1, 1, 2),
 (1, 1, 3),
 (1, 2, 1),
 (1, 2, 2),
 (1, 2, 3),
 (1, 3, 1),
 (1, 3, 2),
 (1, 3, 3),
 (2, 1, 1),
 (2, 1, 2),
 (2, 1, 3),
 (2, 2, 1),
 (2, 2, 2),
 (2, 2, 3),
 (2, 3, 1),
 (2, 3, 2),
 (2, 3, 3),
 (3, 1, 1),
 (3, 1, 2),
 (3, 1, 3),
 (3, 2, 1),
 (3, 2, 2),
 (3, 2, 3),
 (3, 3, 1),
 (3, 3, 2),
 (3, 3, 3)]
```

In [7]:
```python
import itertools

def query(query_sentence, KB, variables):
    """
    For a given query and knowledge base, returns the models where all KB
    sentences are True (or Unknown) in three lists depending on whether
    the query is True, False, or None.

    Args:
        query_sentence: Logic sentence to evaluate
        KB (set): Set of logic sentences
        variables (list/set): Relevant variable names

    Returns:
        (true_models, false_models, unknown_models)
    """

    true_models = []
    false_models = []
    unknown_models = []

    variables = list(variables)

    # Generate every possible truth assignment
    for values in itertools.product([True, False], repeat=len(variables)):

        model = dict(zip(variables, values))

        # Check whether the model is valid for the KB
        valid = True

        for sentence in KB:
```

```
            result = evaluate(sentence, model)

            # KB sentences must not be False
            if result is False:
                valid = False
                break

        if not valid:
            continue

        # Evaluate the query
        query_result = evaluate(query_sentence, model)

        if query_result is True:
            true_models.append(model)
        elif query_result is False:
            false_models.append(model)
        else:
            unknown_models.append(model)

    return true_models, false_models, unknown_models


# ================================================
# TEST 1: Consistent Knowledge Base
# ================================================

KB = {
    Implies("rain", "umbrella"),
    Equals("rain", Not("sunny"))
}

variables = ["rain", "umbrella", "sunny"]

true_models, false_models, unknown_models = query(
    "umbrella",
    KB,
    variables
)

print("=== CONSISTENT KB ===")
print("TRUE MODELS:")
for model in true_models:
    print(model)

print("\nFALSE MODELS:")
for model in false_models:
    print(model)

print("\nUNKNOWN MODELS:")
for model in unknown_models:
    print(model)


# ================================================
# TEST 2: Contradictory KB
# rain AND not rain
# ================================================

KB_contradictory = {
```

```
    "rain",
    Not("rain")
}

variables = ["rain"]

true_models, false_models, unknown_models = query(
    "rain",
    KB_contradictory,
    variables
)

print("\n=== CONTRADICTORY KB 1 ===")
print("TRUE MODELS:", true_models)
print("FALSE MODELS:", false_models)
print("UNKNOWN MODELS:", unknown_models)

if not true_models and not false_models and not unknown_models:
    print("Contradictory KB detected: no valid models exist.")


# ================================================
# TEST 3: Another Contradictory KB
# rain <=> sunny
# rain
# not sunny
# ================================================

KB_contradictory2 = {
    Equals("rain", "sunny"),
    "rain",
    Not("sunny")
}

variables = ["rain", "sunny"]

true_models, false_models, unknown_models = query(
    "rain",
    KB_contradictory2,
    variables
)

print("\n=== CONTRADICTORY KB 2 ===")
print("TRUE MODELS:", true_models)
print("FALSE MODELS:", false_models)
print("UNKNOWN MODELS:", unknown_models)

if not true_models and not false_models and not unknown_models:
    print("Contradictory KB detected: no valid models exist.")
```

```
=== CONSISTENT KB ===
TRUE MODELS:
{'rain': True, 'umbrella': True, 'sunny': False}
{'rain': False, 'umbrella': True, 'sunny': True}

FALSE MODELS:
{'rain': False, 'umbrella': False, 'sunny': True}

UNKNOWN MODELS:

=== CONTRADICTORY KB 1 ===
TRUE MODELS: []
FALSE MODELS: []
UNKNOWN MODELS: []
Contradictory KB detected: no valid models exist.

=== CONTRADICTORY KB 2 ===
TRUE MODELS: []
FALSE MODELS: []
UNKNOWN MODELS: []
Contradictory KB detected: no valid models exist.
```

Testing the querying function

> **Task 7:** Create a knowledge base from the sentences you used to test `evaluate`, query it with the sentences of the same example below, and check that you get the expected results.

No complex function is complete without testing. To do this we will use the sentences from testing our evaluation function as the knowledge base. You will be given a list of "relevant" variables and then you will query the knowledge base with the following sentences with the given expected results.

The query function must follow the stated design requirements (argument and return types).

The solution is accepted only if the querying function exactly returns the stated expected result of each query.

Example 1

- $True$ returns 2 True models (and 0 False/None models).
- $rain \leftrightarrow \neg sunny$ returns 2 True models.
- $play \wedge cloudy$ returns 1 False model, and 1 unknown model.
- $sunny \leftrightarrow play$ returns 2 unknown models.
- $(sunny \leftrightarrow \neg cloudy) \leftrightarrow False$ returns 1 True and 1 False model.

Example 2

- $True$ returns 1 True model (and 0 False/None models).
- $rain \leftrightarrow \neg sunny$ returns 1 True model.
- $storm \wedge cloudy$ returns 1 unknown model.
- $sunny \leftrightarrow storm$ returns 1 unknown model.

- $(sunny \leftrightarrow \neg cloudy) \leftrightarrow False$ returns 1 False model.

Example 3

- $True$ returns 1 True model (and 0 False/None models).
- $windy \leftrightarrow \neg humid$ returns 1 True model.
- $power \wedge generator$ returns 1 unknown model.
- $alert \leftrightarrow generator$ returns 1 unknown model.
- $(alert \leftrightarrow \neg humid) \leftrightarrow False$ returns 1 False model.

The test variables for each example are given below. You are free to test with any (or all) of the examples.

In [8]:
```python
import itertools

print("Variables:", len(variables))
def query(query_sentence, KB, variables):
    '''
    For a given query and knowledge base, returns the models where all KB
    sentences are True or Unknown (None), separated by whether the query
    evaluates to True, False, or None.

    Args:
        query_sentence: Logic sentence to evaluate
        KB (set): Set of logical sentences
        variables (list/set): Relevant variable names

    Returns:
        ([dict], [dict], [dict]):
        (true_models, false_models, unknown_models)
    '''

    true_models = []
    false_models = []
    unknown_models = []

    variables = list(variables)

    # Generate all possible truth assignments
    for assignment in itertools.product([True, False], repeat=len(variables)):

        model = dict(zip(variables, assignment))

        # Check whether this model is valid for the KB
        valid_model = True

        for sentence in KB:
            result = evaluate(sentence, model)

            # KB sentences may be True or None
            # but must not be False
            if result is False:
                valid_model = False
                break

        if not valid_model:
            continue
```

```
        # Evaluate the query in this valid model
        query_result = evaluate(query_sentence, model)

        if query_result is True:
            true_models.append(model)

        elif query_result is False:
            false_models.append(model)

        else:  # None
            unknown_models.append(model)

    return true_models, false_models, unknown_models


KB = {
    Implies("rain", "umbrella"),
    Equals("rain", Not("sunny"))
}

variables = {"rain", "umbrella", "sunny"}

true_models, false_models, unknown_models = query(
    "umbrella",
    KB,
    variables
)

print("TRUE MODELS")
print(true_models)

print("\nFALSE MODELS")
print(false_models)

print("\nUNKNOWN MODELS")
print(unknown_models)
```

```
Variables: 2
TRUE MODELS
[{'umbrella': True, 'rain': True, 'sunny': False}, {'umbrella': True, 'rain': Fal
se, 'sunny': True}]

FALSE MODELS
[{'umbrella': False, 'rain': False, 'sunny': True}]

UNKNOWN MODELS
[]
```

Part 2: Living in the Wumpus World: Logical Agents with propositional logic

In this part we will learn how to use propositional logic and knowledge bases, which you created in Part 1, to create a simple agent to traverse the Wumpus World.

This part includes:

- Implementing an agent that keeps a knowledge base of the Wumpus World and queries it for which tiles are safe to visit.

The content of this lab relates strongly to the material of Chapter 6 (Logical Agents) in the course book (Artificial Intelligence: A Modern Approach). The environment we will tackle in this lab is called the Wumpus World and is covered in the course book as well (Chapter 6.2)

Implementation of the Wumpus World

Below you will find a code implementation of the Wumpus World along with a few helpful definitions and functions. The Wumpus World is fully implemented since this part is about building a propositional logic knowledge base system to build an AI around. However, since the AI will interact with the Wumpus World, being familiar with the code and functions will help you use them later.

In [9]:
```
#Imports
import random
import itertools

# Directions
RIGHT = 0
DOWN  = 1
LEFT  = 2
UP    = 3

# How much the agent will move if it takes a step forward for each direction
direction_step = [(1,0), (0,-1), (-1,0), (0,1)]

# Objects
AGENT   = "a"
WUMPUS  = "w"
PIT     = "p"
GOLD    = "g"

# Actions
NONE = ""
FORWARD = "forward"
TURN_LEFT = "turn_left"
TURN_RIGHT = "turn_right"
GRAB = "grab"
SHOOT = "shoot"
CLIMB = "climb"

class WumpusWorld:

    def __init__(self):
        self.game_over = False
        self.agent_dir = RIGHT
        self.agent_has_arrow = True
        self.agent_has_gold = False
        self.wumpus_screams = False
        self.non_start_squares = list(itertools.product([0,1,2,3],[0,1,2,3]))
        self.non_start_squares.pop(0)
        self.cave = [[[],[],[],[]],[[],[],[],[]],[[],[],[],[]],[[],[],[],[]]]
```

```
        self.cave[0][0].append(AGENT)
        self.agent_pos = (0,0)
        self.random_place(WUMPUS)
        self.random_place(GOLD)
        for square in self.non_start_squares:
            if random.random() < 0.2:
                self.cave[square[0]][square[1]].append(PIT)
        self.performance = 0
        self.last_action = NONE

    def random_place(self,object):
        '''
        Randomly places an object into a tile other than the start.
        Args:
            object (any): The object (typically WUMPUS or GOLD) to place
        '''
        pos = random.choice(self.non_start_squares)
        self.cave[pos[0]][pos[1]].append(object)

    def show(self):
        '''
        Prints the Wumpus World cave and current agent performance.
        '''
        for y in range(3,-1,-1):
            for x in range(0,4):
                [print(o,end="") for o in self.cave[x][y]]
                print("\t",end="")
            print()
        print(f"Performance: {self.performance}")

    def actuate(self, action):
        '''
        If the game is still running, the input action is performed and the game
        Args:
            action (str): The action the agent should perform
        '''
        # If the game is over, nothing can be done
        global hit_squares
        if self.game_over:
            return

        # store the action that was performed, decrease the performance, and rem
        self.last_action = action
        self.performance = self.performance-1
        self.wumpus_screams = False

        # If the NONE action is performed, do nothing
        if action == NONE:
            return

        # FORWARD action: Move the agent one tile in its current direction.
        # End the game and lower performance if it steps into a PIT or WUMPUS
        if action == FORWARD:
            x, y = self.agent_pos
            self.cave[x][y].remove(AGENT)
            xp, yp = direction_step[self.agent_dir]
            x = min(max(x+xp,0),3)
            y = min(max(y+yp,0),3)
            self.agent_pos = (x,y)
            if WUMPUS in self.cave[x][y] or PIT in self.cave[x][y]:
```

```
                self.game_over = True
                self.performance = self.performance-1000
            else:
                self.cave[x][y].append(AGENT)
            return

        # TURN_LEFT action: Change the agents direction 90 degrees counter-clock
        if action == TURN_LEFT:
            self.agent_dir = (self.agent_dir-1)%4
            return
        # TURN_RIGHT action: Change the agents direction 90 degrees clockwise
        if action == TURN_RIGHT:
            self.agent_dir = (self.agent_dir+1)%4
            return
        # GRAB action: If the agent is in the same tile as the gold, remove it f
        if action == GRAB:
            x, y = self.agent_pos
            if GOLD in self.cave[x][y]:
                self.agent_has_gold = True
                self.cave[x][y].remove(GOLD)
            return
        # SHOOT action: If the agent has the arrow,
        #    remove it, decrease performance, and,
        #    if there is a WUMPUS infront of the agent, remove it and add scream
        if action == SHOOT:
            if not self.agent_has_arrow:
                return
            self.agent_has_arrow = False
            self.performance = self.performance-10
            x, y = self.agent_pos
            if self.agent_dir == RIGHT:
                hit_squares = [(min(x+1,3),y),(min(x+2,3),y),(min(x+3,3),y)]
            elif self.agent_dir == DOWN:
                hit_squares = [(x,max(y-1,0)),(x,max(y-2,0)),(x,max(y-3,0))]
            elif self.agent_dir == LEFT:
                hit_squares = [(max(x-1,0),y),(max(x-2,0),y),(max(x-3,0),y)]
            elif self.agent_dir == UP:
                hit_squares = [(x,min(y+1,3)),(x,min(y+2,3)),(x,min(y+3,3))]
            for x, y in hit_squares:
                if WUMPUS in self.cave[x][y]:
                    self.cave[x][y].remove(WUMPUS)
                    self.wumpus_screams = True
            return

        # CLIMB action: If the agent is at the starting tile, end the game and a
        if action == CLIMB:
            if self.agent_pos == (0,0):
                if self.agent_has_gold:
                    self.performance = self.performance+1000
                self.cave[0][0].remove(AGENT)
                self.agent_pos = None
                self.agent_dir = None
                self.game_over = True
            return

    def perceive(self):
        '''
        Returns for each percept whether that percept was sensed after the last
        '''
        x,y = self.agent_pos
```

```
        stench = False
        breeze = False
        for sx,sy in adjacent_tiles(x,y):
            if WUMPUS in self.cave[sx][sy]:
                stench = True
            if PIT in self.cave[sx][sy]:
                breeze = True
        glitter = GOLD in self.cave[x][y]
        bump = False
        if self.last_action == FORWARD:
            if self.agent_dir == RIGHT and x == 3:
                bump = True
            elif self.agent_dir == DOWN and y == 0:
                bump = True
            elif self.agent_dir == LEFT and x == 0:
                bump = True
            elif self.agent_dir == UP and y == 3:
                bump = True
        scream = self.wumpus_screams
        return(stench, breeze, glitter, bump, scream)

def adjacent_tiles(x, y):
    '''
    Helper function that, given a position, returns all tiles that are adjacent
    Args:
        x (int): The x coordinate of the position
        y (int): The y coordinate of the position
    '''
    tiles = []
    for sx,sy in [(x-1,y),(x+1,y),(x,y-1),(x,y+1)]:
        if sx<4 and sx>-1 and sy<4 and sy>-1:
            tiles.append((sx,sy))
    return tiles
```

Using logic to understand the Wumpus World

Now that we have a working Propositional Logic, Knowledge Base, and Query system in place, we can finally start using it to create an agent. Knowledge-based agents can come in many forms, but the most typical behavior is to dump all the relevant knowledge of the world into a knowledge base and then query the knowledge base for what action to perform next. The percepts from the environment are then used to add new knowledge into the knowledge base to help with the next query.

Doing all of this can require very extensive knowledge bases and complex agents, which is beyond the scope of this assignment. Instead we will use a very simple procedure for our agent:

- If the agent has the gold we know there must be a safe path back to the entrance, take actions to follow that path back and then use CLIMB to finish the game.
- If there is glitter on the current tile, use the GRAB action to pick it up.
- Otherwise, query every tile for whether it's safe or not.
  - If there is a safe tile that has not yet been visited, calculate a path to it and then follow that path.
    - If there is no path, calculate a path to the next unvisited safe tile.

- If all safe tiles have been visited or are unreachable, find a path back to the start tile and then use CLIMB to finish the game.

As you can see this agent is very simple, not even considering using the SHOOT action, and treats every tile that it doesn't know for sure to be safe as unsafe. For now though, this is enough. Before we can start creating an agent, we need a knowledge base for it to query. There are many things we know about the Wumpus World, many of which we could turn into logic and put into the knowledge base. However, the more things we put in the knowledge base the longer the query will take, so we want to stick with only the essentials. For example, we know that there can only be one WUMPUS and that each tile (except the start) has a 20% chance of being a PIT. But the 20% chance is difficult to encode in propositional logic and not really relevant to our query, since we only care about things we know for sure.

So what do we put in the knowledge base? We want to look for safe squares, so we need to put a sentence for each tile defining what it means for that tile to be safe (there is no WUMPUS and no PIT there). We also need some way to figure out whether there is a WUMPUS on a tile, so we need to add that a tile has stench if and only if there is a WUMPUS on at least one of its adjacent tiles. The same goes for PIT and breeze. Of course we also know some things about the starting tile.

Then we need a way to add new knowledge into the knowledge base based on the percepts that are encountered.

So now we need to create two functions. One that returns the knowledge base about all relevant things we know at the start of each game, and one that given the percepts, the agent's position when those percepts were received, and a knowledge base, adds new relevant knowledge to the knowledge base. A skeleton for each function is provided below.

> **Task 8:** Implement `initialWumpusKB()`, which returns a knowledge base with everything known at the start of each game, and `addWumpusKnowledge()`, which adds new knowledge to the knowledge base based on the agent's percepts and position.

Hint: Remember that each sentence needs to be created for each tile and that each tile needs its own versions of all symbols. Try to think of a systematic way to name the symbols based on their tile coordinates.

In [10]:
```
def initialWumpusKB():
    '''
    Returns a knowledge base of sentences known to be true in all initial Wumpus
    Returns ({any}): A set of logic sentences
    '''

    KB = set()

    for x in range(4):
        for y in range(4):
            pit = f"pit_{x}_{y}"
```

```
            wumpus = f"wumpus_{x}_{y}"
            breeze = f"breeze_{x}_{y}"
            stench = f"stench_{x}_{y}"
            safe = f"safe_{x}_{y}"

            # Safe square definition
            KB.add(
                Equals(
                    safe,
                    And(
                        Not(pit),
                        Not(wumpus)
                    )
                )
            )

            # Breeze iff adjacent pit
            adjacent_pits = [
                f"pit_{ax}_{ay}"
                for ax, ay in adjacent_tiles(x, y)
            ]

            KB.add(
                Equals(
                    breeze,
                    Or(*adjacent_pits)
                )
            )

            # Stench iff adjacent wumpus
            adjacent_wumpus = [
                f"wumpus_{ax}_{ay}"
                for ax, ay in adjacent_tiles(x, y)
            ]

            KB.add(
                Equals(
                    stench,
                    Or(*adjacent_wumpus)
                )
            )

    # Known facts about starting square
    KB.add(Not("pit_0_0"))
    KB.add(Not("wumpus_0_0"))
    KB.add("safe_0_0")

    return KB


def addWumpusKnowledge(percepts, agent_pos, KB):
    '''
    Updates the given knowledge base by adding (and potentially removing)
    sentences from the knowledge base, based on the Wumpus World
    agent's position and perceptions.

    Args:
        percepts ([bool]): A list of whether each percept is active
        agent_pos (int, int): The tile currently occupied by the agent
        KB ({any}): The knowledge base of the current Wumpus World
```

```
    '''

    stench, breeze, glitter, bump, scream = percepts
    x, y = agent_pos

    # Current square is known safe because agent survived entering it
    KB.add(Not(f"pit_{x}_{y}"))
    KB.add(Not(f"wumpus_{x}_{y}"))
    KB.add(f"safe_{x}_{y}")

    # Stench knowledge
    if stench:
        KB.add(f"stench_{x}_{y}")
    else:
        KB.add(Not(f"stench_{x}_{y}"))

    # Breeze knowledge
    if breeze:
        KB.add(f"breeze_{x}_{y}")
    else:
        KB.add(Not(f"breeze_{x}_{y}"))

    # Optional percepts
    if glitter:
        KB.add(f"glitter_{x}_{y}")
    else:
        KB.add(Not(f"glitter_{x}_{y}"))

    if bump:
        KB.add(f"bump_{x}_{y}")

    if scream:
        KB.add("wumpus_dead")
```

In [11]:

```
# ================================================
# TASK 8 - ESSENTIAL TESTS
# ================================================

print("TEST 1: Initial KB")

KB = initialWumpusKB()

print(Not("pit_0_0") in KB)      # Expected: True
print(Not("wumpus_0_0") in KB)   # Expected: True
print("safe_0_0" in KB)          # Expected: True


print("\nTEST 2: No stench, no breeze at start")

KB = initialWumpusKB()

addWumpusKnowledge(
    (False, False, False, False, False),
    (0, 0),
    KB
)

print(Not("stench_0_0") in KB)   # Expected: True
print(Not("breeze_0_0") in KB)   # Expected: True
print("safe_0_0" in KB)          # Expected: True
```

```
print("\nTEST 3: Breeze perception")

KB = initialWumpusKB()

addWumpusKnowledge(
    (False, True, False, False, False),
    (0, 0),
    KB
)

print("breeze_0_0" in KB)        # Expected: True


print("\nTEST 4: Stench perception")

KB = initialWumpusKB()

addWumpusKnowledge(
    (True, False, False, False, False),
    (0, 0),
    KB
)

print("stench_0_0" in KB)        # Expected: True


print("\nTEST 5: Glitter perception")

KB = initialWumpusKB()

addWumpusKnowledge(
    (False, False, True, False, False),
    (0, 0),
    KB
)

print("glitter_0_0" in KB)       # Expected: True


print("\nTEST 6: Scream perception")

KB = initialWumpusKB()

addWumpusKnowledge(
    (False, False, False, False, True),
    (0, 0),
    KB
)

print("wumpus_dead" in KB)       # Expected: True
```

```
TEST 1: Initial KB
True
True
True

TEST 2: No stench, no breeze at start
True
True
True

TEST 3: Breeze perception
True

TEST 4: Stench perception
True

TEST 5: Glitter perception
True

TEST 6: Scream perception
True
```

Creating the agent

With knowledge base in hand it's time to create our simple naive agent.

Reading the description of the agent above, you probably noticed that it requires a search algorithm to find a path from one point to another. Since search algorithms aren't covered in this course a simple implementation of the shortest path algorithm **A*** has been implemented in the next code block for you to use. It takes the start tile, goal tile, and a list of all safe tiles and then returns the next tile to walk to. Even though search-algorithms like **A*** are not covered in this course, you can read more about them in Chapter 3 (Solving Problems by Searching) of the course book (Artificial Intelligence: A Modern Approach).

You also receive a function to run the game given a class to return actions (an agent), and a skeleton for the agent you will implement based on the earlier description.

> **Task 9:** Implement `SimpleAgent` (skeleton below) following the procedure described in *Using logic to understand the Wumpus World*.
>
> **Important:** You must use the `query()` function created in Part 1 to solve Part 2.

Hint: Querying with every symbol of the Wumpus World as variables means checking an enormous number of models. Think about which few variables are actually relevant for whether a single tile is safe.

In [12]:
```python
# An implementation of a priority queue that is necessary for calculating A*
import heapq

def a_star_on_tiles(start, goal, safe_tiles):
    '''
    Takes a start tile, end tile, and a list of safe tiles and returns the next
```

```
    Assumes that the world is a grid where each tile is connected to the adjacen
    Args:
        start (int, int): The coordinates of the start tile
        goal (int, int): The tile to find a shortest path towards
        safe_tiles [(int, int)]: A list of the tiles that can be traversed (shou
    Returns ((int, int)/None): The next tile in the shortest path if one exists
    '''
    frontier = []
    heapq.heappush(frontier, (0, start))
    came_from = {}
    cost_so_far = {}
    came_from[start] = None
    cost_so_far[start] = 0

    # Run A* to get a dictionary containing for each tile, the previous tile alo
    while len(frontier) > 0:
        distance, (x, y) = heapq.heappop(frontier)

        if (x,y) == goal:
            break

        for neighbor in [n for n in adjacent_tiles(x,y) if n in safe_tiles]:
            new_cost = cost_so_far[(x,y)] + 1 #Can be adapted to handle turning
            if neighbor not in cost_so_far or new_cost < cost_so_far[neighbor]:
                cost_so_far[neighbor] = new_cost
                priority = new_cost + abs(x-neighbor[0]) + abs(y-neighbor[1])
                heapq.heappush(frontier, (priority, neighbor))
                came_from[neighbor] = (x,y)

    # Walk backward along the path until the first tile beyond the start is foun
    # If the start tile is unreachable, return None
    current = goal
    while True:
        if current not in came_from:
            return None
        if came_from[current] == start:
            break
        current = came_from[current]

    return current
```

In [13]:
```
def play_game(agent, show=True):
    '''
    Initiates and runs a random Wumpus World by calling the Agent.act() function
    Args:
        agent (any): An object which has implemented an act function
        show (bool): Whether to print the states of the game or not
    '''
    w = WumpusWorld()
    while not w.game_over:
        if show:
            w.show()
        w.actuate(agent.act(w.perceive(), w.agent_pos, w.agent_dir))
    if show:
        w.show()
```

In [14]:
```
class SimpleAgent():
    '''
    Simple logical agent for the Wumpus World.
```

```
    '''

    def __init__(self):

        self.KB = initialWumpusKB()
        self.visited = set()
        self.has_gold = False

    def move_towards(self, current_pos, current_dir, target_pos):

        x, y = current_pos
        tx, ty = target_pos

        dx = tx - x
        dy = ty - y

        if dx == 1:
            desired_dir = RIGHT
        elif dx == -1:
            desired_dir = LEFT
        elif dy == 1:
            desired_dir = UP
        else:
            desired_dir = DOWN

        if current_dir == desired_dir:
            return FORWARD

        if (desired_dir - current_dir) % 4 == 1:
            return TURN_RIGHT

        return TURN_LEFT

    def act(self, percepts, position, direction):
        '''
        Returns an action based on current percepts,
        position and direction.
        '''

        self.visited.add(position)

        # Update KB with current percepts
        addWumpusKnowledge(
            percepts,
            position,
            self.KB
        )

        stench, breeze, glitter, bump, scream = percepts

        # Grab gold immediately
        if glitter:
            self.has_gold = True
            return GRAB

        # -------------------------------------------------
        # Determine which tiles are definitely safe
        # -------------------------------------------------
        safe_tiles = []
```

```
        for x in range(4):
            for y in range(4):

                # Only use variables relevant for this tile
                variables = {
                    f"safe_{x}_{y}",
                    f"pit_{x}_{y}",
                    f"wumpus_{x}_{y}"
                }

                for ax, ay in adjacent_tiles(x, y):
                    variables.add(f"breeze_{ax}_{ay}")
                    variables.add(f"stench_{ax}_{ay}")

                true_models, false_models, unknown_models = query(
                    f"safe_{x}_{y}",
                    self.KB,
                    variables
                )

                # Safe only if no valid model makes it unsafe
                if (
                    len(true_models) > 0 and
                    len(false_models) == 0 and
                    len(unknown_models) == 0
                ):
                    safe_tiles.append((x, y))

        # Always consider visited tiles safe
        for tile in self.visited:
            if tile not in safe_tiles:
                safe_tiles.append(tile)

        # -------------------------------------------------
        # If carrying gold, go home
        # -------------------------------------------------
        if self.has_gold:

            if position == (0, 0):
                return CLIMB

            next_tile = a_star_on_tiles(
                position,
                (0, 0),
                safe_tiles
            )

            if next_tile is not None:
                return self.move_towards(
                    position,
                    direction,
                    next_tile
                )

        # -------------------------------------------------
        # Explore unvisited safe tiles
        # -------------------------------------------------
        for tile in safe_tiles:

            if tile in self.visited:
```

```
                continue

            next_tile = a_star_on_tiles(
                position,
                tile,
                safe_tiles
            )

            if next_tile is not None:
                return self.move_towards(
                    position,
                    direction,
                    next_tile
                )

        # ------------------------------------------------
        # No new safe tiles -> return home
        # ------------------------------------------------
        if position == (0, 0):
            return CLIMB

        next_tile = a_star_on_tiles(
            position,
            (0, 0),
            safe_tiles
        )

        if next_tile is not None:
            return self.move_towards(
                position,
                direction,
                next_tile
            )

        return CLIMB
```

Note: The simple agent will not always find the gold. Sometimes the gold cannot be reached safely (for example when it is surrounded by pits, or when there is already a breeze or stench on the start tile), and the agent will then climb out without it. In some cases the agent might also end up looping forever.

In these cases it's fine to stop the cell and reset the simulation by running it again, or to test with a pre-set environment. Calling `random.seed(...)` with a fixed number before `play_game` gives the same world every time, e.g.:

```
random.seed(3)
play_game(SimpleAgent())
```

In [15]:
```
# Execute this cell to play one game in the Wumpus World with the SimpleAgent
random.seed(3)
play_game(SimpleAgent())

#random.seed(1)
#play_game(SimpleAgent())

#random.seed(5)
#play_game(SimpleAgent())
```

```
                        p
        p       g
        p
a       w
Performance: 0
                        p
        p       g
        p
        w
Performance: -1
```